

UNIT-01

RANKA	
DATE	/ /
PAGE	

Introduction to data science

Data science is an interdisciplinary field that uses scientific methods, statistics, mathematics, programming and domain knowledge to collect, process, analyze and interpret data to extract meaningful insights and support decision-making.

Key components of data science

- (i) Data Collection: Gathering data from sources such as databases, websites, sensors or surveys.
- (ii) Data cleaning: Removing errors, duplicates and inconsistencies to improve data quality.
- (iii) Data Analysis: Exploring data using statistical and analysis techniques to identify patterns & trends.
- (iv) Data Visualization: Presenting data through charts, graphs, and dashboards to communicate findings effectively.
- (v) Machine Learning: Building models that learn from data to make predictions or automate decisions.
- (vi) Decision-making: Using insights from data to solve business or research problems.

Applications of data science

Healthcare, Finance, E-commerce, Education, Transportation

Facts of Data

* It refers to the different characteristics, dimensions or aspects of data that determine its quality, structure, usability and value.

* Understanding these facts helps in collecting, managing, analyzing and interpreting data effectively.

(i) Volume: Refers to the amount of data generated & stored.

Ex: Social media posts generate millions of data everyday.

(ii) Variety: Refers to the different types and format of data.

Types of Data:

* Structured Data (Databases, Excel sheets)

* Semi-Structured Data (JSON, XML)

* Unstructured Data (Image, videos, Email)

(iii) Velocity: Refers to the speed at which data is generated, collected and processed.

* High velocity requires real-time or near-real time processing.

Ex: Stock market transactions.

Online payment systems.

(iv) Venacity:- Refers to the accuracy, reliability & trustworthiness of data.

Ex:- Removing duplicate customer records before analysis

(v) Value:- Refers to the usefulness of data in generating meaningful insights and supporting decision-making.

Ex:- Sales data used to predict customer buying behaviours.

(vi) Validity:- Ensures that data follows the required rules, formats and constraints.

* Invalid data can cause processing errors and inaccurate results.

Ex:- A student's age can't be negative.

(vii) Variability:- Refers to the inconsistency or changing nature of data over time.

Ex:- Seasonal changes in electricity demand.

(viii) Visualization:- Represents data using graphs, charts and dashboards to make it easier to understand and interpret.

Ex:- Bar charts
Line charts
Pie charts.

Data Science Process

- * The Data Science Process is a systematic sequence of steps used to transform raw data into meaningful insights and predictive models.
- * It helps organization solve real-world problems through data driven decision-making.

Steps in Data Science Process

(i) Problem Definition:- Clearly identify the problem or business objective.

- * Define the goals and expected outcomes.

- * Understand stakeholders requirements

(ii) Data Collection:- Gather data from various sources such as Databases, Websites, APIs, IoT Devices.

(iii) Data Cleaning and Preprocessing:- Improve data quality by removing errors and inconsistencies.

Tasks:-

- Handle missing values
- Remove duplicate records
- Detect and remove outliers
- Correct inconsistent data
- Normalize or standardize data

(iv) Exploratory Data Analysis (EDA):- Analyze the cleaned data to discover patterns and relationships.

Tools:- Histograms, Scatter plots, Heatmaps.

(v) Feature Engineering:- Create or modify variables to improve model performance.



(vi) Model Building:- Select and train suitable machine learning algorithms

Common Algorithms:-
Linear Regressions
Decision Tree
Random Forest

(vii) Model Evaluations:- Measure the model's performance using appropriate evaluation metrics.

Classification Metrics:-
Accuracy
Precision
Recall
F1-Score.

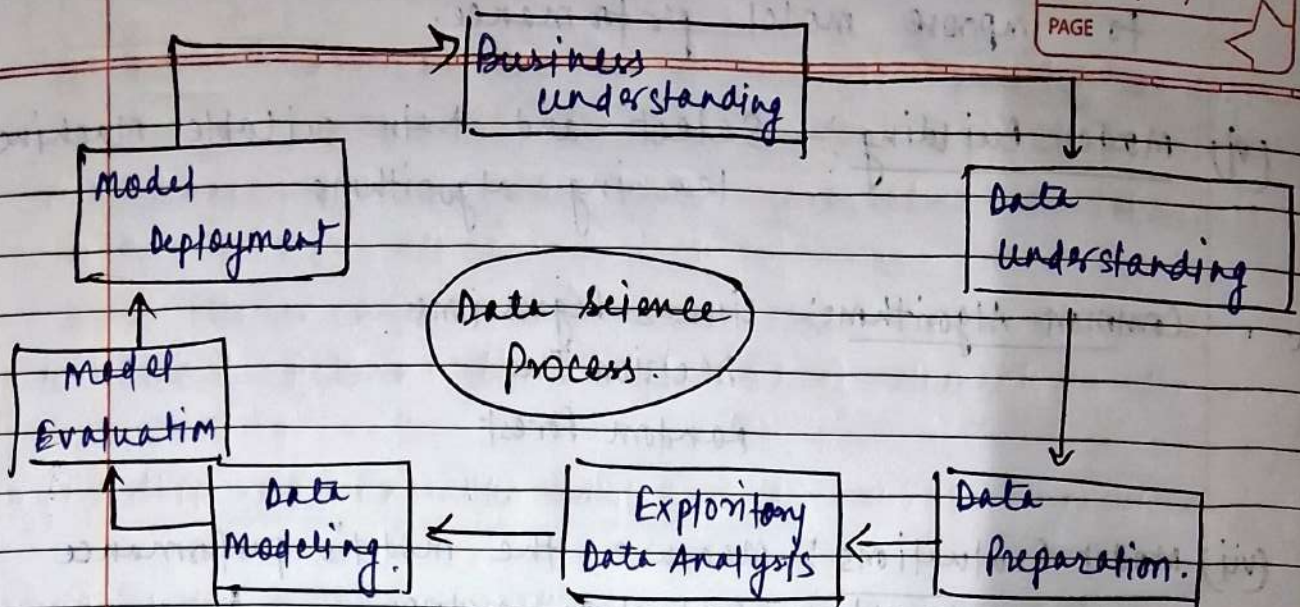
(viii) Model Deployment:- Deploy the trained model into a real-world environment.

* Integrate it with applications or business systems.

(ix) Monitoring and Maintenance:- continuously monitor the model's performance.

* Update or retrain the model when new data becomes available.

* Ensures the model remains accurate over time.



Introduction to Numpy

- * Numpy is a python library that enables fast and efficient numerical operations on array and matrices.
- * It is widely used because it is faster and more memory efficient than python lists for numerical computations.
- * It supports 1D, 2D and multidimensional arrays.
- * Provides fast execution due to optimized c-based implementation.
- * Uses less memory than python list
- * Supports vectorized operations with explicit loops

Applications of Numpy:

- * Data Science & Data Analysis
- * Machine Learning
- * AI
- * Image Processing
- * Statistical Computing
- * Signal Processing

Installation :- pip install numpy.

RANKA	
DATE	/ /
PAGE	

Importing Numpy :- import numpy as np.

Creating Arrays :-

Creating Numpy Array Objects

* One Dimensional Array :- import numpy as np
arr = np.array([10, 20, 30, 40, 50])

print(arr)

* Two - Dimensional Array :- arr2 = np.array([10, 20, 30],
[40, 50, 60])

print(arr2)

* Three Dimensional Array :- arr3 = np.array([1, 2, 3],
[4, 5, 6],
[7, 8, 9])

print(arr3)

print(arr.shape)

(5,)

~~print~~ len(arr.shape)

1

print(arr2.shape)
(2, 2)

print(arr3.shape)
(3, 3)

len(arr2.shape)
2

len(arr3.shape)
3

arr4 = np.array([1, 2],
[3, 4],
[5, 6])

print(arr4.shape)

(3, 2)

Attributes in Numpy

- (i) ndim:- Returns the no. of dimensions (axes) of array.
- (ii) shape:- Returns a tuple representing the dimensions of array.
- (iii) size:- Returns the total no. of elements in the array.
- (iv) dtype:- Returns the datatype of the array elements.
- (v) itemsize:- Returns the size (in bytes) of each array element.
- (vi) nbytes:- Returns the total memory occupied by the array in bytes.
- (vii) T (Transpose):- Returns the transpose of the array.

* import numpy as np.

```
arr = np.array([1, 2, 3],  
               [4, 5, 6])
```

```
print("Dimensions:", arr.ndim)
```

```
print("Shape:", arr.shape)
```

```
print("Size:", arr.size)
```

```
print("dtype:", arr.dtype)
```

```
print("Item Size:", arr.itemsize)
```

```
print("Total bytes:", arr.nbytes)
```

Dimensions: 2

shape: (2, 3)

Size: 6

dtype: int64

Item Size: 8

Total bytes: 48

```
* import numpy as np
arr = np.array([1, 2],
               [3, 4])
print(arr.T)
```

```
[[1, 3]
 [2, 4]]
```

```
* import numpy as np
a = [1, 2, 3, 4]
```

```
arr1 = np.array(a)
```

```
arr1.dtype
```

```
dtype('int32')
```

```
arr2 = np.array(a, dtype=np.int64)
```

```
arr2.dtype
```

```
dtype('int64')
```

```
b = [1.2, 3.4, 5.]
```

```
type(b[0])
```

```
float
```

```
arr3 = np.array(b)
```

```
print(arr3)
```

```
array([1.2, 3.4, 5.])
```

```
arr3.astype(int)
```

```
print(arr3)
```

```
array([1, 3, 5])
```

Numpy Array Objects:- A numpy array object is the main data structure in Numpy.

DATE

PAGE

* It is called an nd array (N-dimensional array)

* An ndarray stores elements of same data type in a multidimensional structure, making numerical computations fast and memory efficient

or An numpy array object (ndarray) is collection of elements of the same data type arranged in one or more dimensions (1D, 2D, 3D, etc).

* Creating array using np.zeros() and np.ones()

```
import numpy as np
```

```
arr1 = np.zeros((2,3))
```

```
arr2 = np.ones((3,2))
```

```
print(arr1)
```

```
print(arr2)
```

```
[[0. 0. 0.]  
 [0. 0. 0.]]
```

```
[[1. 1.]  
 [1. 1.]  
 [1. 1.]]
```

* Creating array using np.empty()

```
import numpy as np
```

```
arr = np.empty((2,2))
```

```
print arr
```

```
[[6.9e-310 6.9e-310]
```

```
 [6.9e-310 6.9e-310]]
```

* Creating array using np.arange()

```
import numpy as np.  
arr1 = np.arange(1, 11)
```

```
print(arr1)
```

```
[1 2 3 4 5 6 7 8 9 10]
```

```
arr2 = np.arange(0, 20, 2)
```

```
print(arr2)
```

```
[0 2 4 6 8 10 12 14 16 18]
```

* Creating array using np.linspace()

Creates evenly spaced numbers b/w two values.

```
import numpy as np.
```

```
arr = np.linspace(1, 10, 5) # Print 5 no.s b/w 1 and 10  
print(arr)                  with equal spacing.
```

```
[1.  3.25 5.5  7.75 10.]
```

* Creating an array using np.eye()

Creates an identity matrix

```
import numpy as np
```

```
arr = np.eye(3)
```

```
print(arr)
```

```
[[1.  0.  0.]
```

```
 [0.  1.  0.]
```

```
 [0.  0.  1.]
```

* Creating an array with random values

```
import numpy as np
```

```
arr1 = np.random.rand((2,3))  
print(arr1)
```

$\begin{bmatrix} 0.54 & 0.81 & 0.29 \\ 0.18 & 0.73 & 0.95 \end{bmatrix}$

```
arr2 = np.random.randint(1, 10, (2,3))  
print(arr2)
```

$\begin{bmatrix} 4 & 8 & 1 \\ 6 & 2 & 9 \end{bmatrix}$

```
arr3 = np.random.randint(5)  
print(arr3)
```

4

```
arr4 = np.random.randint(5, size=(2,2))  
print(arr4)
```

$\begin{bmatrix} 0 & 2 \\ 1 & 4 \end{bmatrix}$

Basic operations on Numpy

(i) Arithmetic operations

```
import numpy as np.
```

```
a = np.array([10, 20, 30])
```

```
b = np.array([1, 2, 3])
```

```
print(a+b) # Addition  
[11, 22, 33]
```

print(a-b) # Subtraction

[9 18 27]

print(a*b) # Multiplication

[10 40 90]

print(a/b) # Division

[10. 10. 10.]

e = np.array([2, 3, 4])

print(a**b) # Exponentiation

[4 9 16]

(i) Scalar operations: It applies the same value to every element.

import numpy as np

arr = np.array([1, 2, 3])

print(arr+5) → [6 7 8]

print(arr*2) → [2 4 6]

(ii) Reshaping Arrays:

import numpy as np

arr = np.arange(12)

new = arr.reshape(3, 4)

print(new)

[0 1 2 3]

[4 5 6 7]

[8 9 10 11]

(3, 4) array

[0 1 2 3]

[4 5 6 7]

[8 9 10 11]

(iv) Aggregate operations:-

```
import numpy as np
```

```
arr = np.array([10, 20, 30])
```

```
print(np.sum(arr)) # Returns sum of all elements  
60
```

```
print(np.mean(arr)) # Return mean of the array.  
20.0
```

```
print(np.max(arr)) # Gives maxm element in the array  
30
```

```
print(np.min(arr)) # " minm " " " " "  
10
```

(v) Comparison Operations

```
import numpy as np
```

```
arr = np.array([5, 10, 15])
```

```
print(arr > 8)
```

```
[False True True]
```

(vi) Transpose and Multiplication of Arrays

```
import numpy as np
```

```
arr = np.array([[1, 2, 3],  
                [4, 5, 6]])
```

```
print(arr.T)
```

```
[[1, 4]
```

```
 [2, 5]
```

```
 [3, 6]]
```

`A = np.array([[1, 2], [3, 4]])`

`B = np.array([[5, 6], [7, 8]])`

`print (np.dot(A, B))` OR `A @ B`

`[[19 22] [43 50]]`

* For multiplication of matrix $(m \times n) (p \times q) \rightarrow n = p$.

(vii) Sorting an Array:-

`import numpy as np`

`arr = np.array([30, 10, 20])`

`print (np.sort(arr))`

`[10 20 30]`

* Sorting in 3D array:-

A 3D array consists of multiple 2D (matrices) array.

In numpy `np.sort()` function sorts the elements of an array along a specified axis.

By default it sorts along the last axis (`axis=-1`)

`axis=0`: Sorts along the first axis (depth)

`axis=1`: Sorts along the second axis (rows)

`axis=2`: Sorts along the third axis (columns)

`axis=-1`: Sorts along the last axis (columns)

import numpy as np.

```
arr1 = np.array([[[[3, 1, 2],  
[4, 7, 8]],  
[[6, 4, 5],  
[12, 10, 11]]])
```

```
sorted_arr1 = np.sort(arr1) → axis = -1 (by default)  
print(sorted_arr1)
```

```
[[[1 2 3]  
[7 8 9]]  
[[4 5 6]  
[10 11 12]]]
```

```
arr2 = np.array([[[[3, 1],  
[8, 6]],  
[[2, 4],  
[5, 7]]]])  
print(np.sort(arr2, axis=0))
```

```
[[[2 4]  
[5 6]]]
```

```
[[[3 4]  
[8 7]]]
```

```
arr3 = np.array([[[[5, 2],  
[1, 4]],  
[[9, 6],  
[7, 8]]]])  
print(np.sort(arr3, axis=1))
```

```
[[[1 2]  
[5 4]]  
[[7 6]  
[9 8]]]
```

axis=0 → column wise (top to bottom)

axis=1 → row-wise (left to right).

RANKA	
DATE	/ /
PAGE	

Ex1 ~~axis=-1 → sort along last axis (for 2D same as axis=1).~~

import numpy as np

a = np.array([[9, 3, 7], [2, 8, 5], [6, 1, 4]])

↓
a = np.array([[9, 3, 7],
[2, 8, 5],
[6, 1, 4]])

print(np.sort(a, axis=0))

[[2 1 4]

[6 3 5]

[9 8 7]]

b = np.sort(a, axis=1)

print(b)

[[3 7 9]

[2 5 8]

[1 4 6]]

* Sort an array in descending order

arr = np.array([50, 60, 20, 40, 90])

result = np.sort(arr)[::-1]

print(result)

[90 60 50 40 20]

Joining Arrays

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

result = np.concatenate((a, b))
print(result)
```

[1 2 3 4 5 6]

```
c = np.array([[1, 2], [3, 4]])
d = np.array([[5, 6], [7, 8]])
```

```
result = np.concatenate((c, d))
print(result)
```

[[1 2] [3 4] [5 6] [7 8]]

```
print(np.concatenate((c, d), axis=0))
```

[[1 2] [3 4] [5 6] [7 8]]

```
print(np.concatenate((c, d), axis=1))
```

[[1 2 5 6]

[3 4 7 8]]

Stack → Combining arrays by adding new axis

hstack → join arrays side by side (column)

vstack → put arrays below one another (row)

```
import numpy as np
a = np.array([1, 2])
b = np.array([3, 4])
```

```
print(np.hstack(a, b))
[[1 2]
 [3 4]]
```

```
print(np.hstack(a, b))
[1 2 3 4]
```

```
print(np.vstack(a, b))
[[1 2]
 [3 4]]
```

Splitting Arrays

Dividing one array into multiple smaller arrays.

(i) np.array-split() :- Can split array into unequal sized parts.

import numpy as np.

```
arr = np.array([1, 2, 3, 4, 5, 6])
```

```
print(np.array_split(arr, 3))
```

```
[array([1, 2]), array([3, 4]), array([5, 6])]
```

```
arr2 = np.array([1, 2, 3, 4, 5])
```

```
print(np.array_split(arr, 3))
```

```
[array([1, 2]), array([3, 4]), array([5])]
```

(ii) np.split() :- Divides an array into equal parts.

import numpy as np.

```
arr1 = np.array([1, 2, 3, 4, 5, 6])
```

```
print(np.array np.split(arr1, 3))
```

```
[array([1, 2]), array([3, 4]), array([5, 6])]
```

```
arr2 = np.array([1, 2, 3, 4, 5])
```

```
print(np.split(arr1, 3))
```

ValueError:

np.split → split the array column wise

np.vsplit → split the array row wise

Searching in numpy array.

(i) np.where(): Returns the index / position where the condition is true.

```
import numpy as np.
```

```
arr = np.array([10, 20, 30, 40, 50])
```

```
print(np.where(arr == 30))
```

(array([2]),) → 30 is present at index 2.

```
print(np.where(arr > 25))
```

(array([2, 3, 4]),) → 2, 3, 4 indices satisfies the condⁿ.

(ii) np.searchsorted(): used to find out where an element should be inserted in a sorted array.

```
import numpy as np.
```

```
arr = np.array([10, 20, 30, 40, 50]).
```

```
result = np.searchsorted(arr, 35)
```

```
print(result)
```

3 → Bcoz 35 should be inserted at index 3.

Adding element to an array.

(i) np.append(): Add elements at the end.

```
import numpy as np  
arr = np.array([1, 2, 3, 4])  
arr = np.append(arr, 5)  
print(arr)
```

[1 2 3 4 5]

```
arr = np.append(arr, [6, 7, 8])  
print(arr)
```

[1 2 3 4 5 6 7 8]

(ii) np.insert(): Add element at a specific position.

```
import numpy as np  
arr = np.array([1, 2, 4, 5])
```

```
arr = np.insert(arr, 2, 3)
```

```
print(arr)
```

Index

Element

[1 2 3 4 5]

Numpy Indexing

It is used to access individual elements or groups of element from an array.

* import numpy as np.

```
arr = np.array([10, 20, 30, 40, 50])
```

```
print(arr[0]) # 10
```

Index (+): 0 1 2 3 4

```
print(arr[2]) # 30
```

```
print(arr[-1]) # 50
```

10 20 30 40 50

```
print(arr[-5]) # 10
```

```
print(arr[0]) # 10
```

Index (-): -5 -4 -3 -2 -1

* Indexing in 2D array.

```
arr = np.array([[10, 20, 30],  
                [40, 50, 60],  
                [70, 80, 90]])
```

Row
↓
column

General syntax: array [row, column]

```
print(arr[0, 0]) # 10
```

```
print(arr[1, 2]) # 60
```

Row Indexing: array [row-index]

```
print(arr[0]) # [10 20 30]
```

```
print(arr[1]) # [40 50 60]
```

```
print(arr[2]) # [70 80 90]
```

```
print(arr[1, :]) # [40 50 60]
```

Column indexing:- array[:, column-index].

~~print(arr[:, 1]) # [10 40 70]~~

~~print(arr[:, 2]) # [20 50 80]~~

~~print(arr[:, 3]) # [30 60 90]~~

~~print(arr[:, -1]) # [30 60 90]~~

* arr = np.array([[10, 20, 30],
[40, 50, 60],
[70, 80, 90]])

print(arr[[0, 2]]) # select rows 0 and 2.

[[10 20 30]
[70 80 90]]

* print(arr[:, [0, 2]]) # select columns 0 and 2.

[[10 30]
[40 60]
[70 90]]

Boolean Indexing.

arr = np.array([10, 25, 30, 45, 50])

print(arr[arr > 30]) # [45 50]

print(arr[arr < 30]) [10 25]

print(arr[arr == 30]) [30]

Multiple condition indexing

import numpy as np.

* arr = np.array([10, 20, 30, 40, 50, 60])

(1D)

print(arr[(arr > 20) + (arr < 50)])

[30 40]

print(arr[(arr < 20) | (arr > 50)])

[10 60]

print(arr[~(arr > 30)])

[10 20 30]

* arr2 = np.array([[10, 20, 30],

(2D)

[40, 50, 60],

[70, 80, 90]])

print(arr2[arr2 > 50])

[60 70 80 90]

print(arr2[[0, 2], [1, 2]])

[20 90]

* arr3 = np.array([[

(3D)

[1, 2, 3],

[4, 5, 6],

[7, 8, 9],

[10, 11, 12]])

print(arr3.shape)

Rows

(2, 2, 3)

↓
H.S. of
blocks

Columns

print(arr3[0, 1, 2])

6

DATE / /

PAGE

It works on 1D, 2D, 3D and higher-dimensional array.

start $\rightarrow$ Index from where solving begins.

step $\rightarrow$ gap b/w the selected elements.

arr = np. array ([10, 20, 30, 40, 50, 60]) $\rightarrow$ 1D
arr
0 1 2 3 4 5

```
print(arr[ : 4]) # [10 20 30 40]
```

```
arr
print(arr[1:]) # [20 30 40 50 60]
```

```
arr
↑
print(arr[0:4]) # [10 20 30 40]
```

`print(arr[1:5])` # [20 30 40 50]

```
print(arr[2:]) # [30, 40 50 60]
```

```
print([1::2]) # [10 30 50]
```

```
print(aan[::2] * [10, 40])
```

```
print(arr[-3:]) # [40 50 60]
```

Slicing in a 2D array.

Syntax: array [row-slice, column-slice]

import numpy as np.

```
arr = np.array([[10, 20, 30, 40],  
               [50, 60, 70, 80],  
               [90, 100, 110, 120]])
```

```
print(arr.shape)
```

```
(3, 4)
```

```
print(arr[0, :])
```

→ 4 means all the columns.

```
[10 20 30 40]
```

```
print(arr[:2, :])
```

Two rows & all columns.

```
[[10 20 30 40]
```

```
 [50 60 70 80]]
```

```
print(arr[:, 0])
```

selecting columns.

```
[10 50 90]
```

```
print(arr[:, :2])
```

first two columns.

```
[[10 20]
```

```
 [50 60]
```

`print(arr[::-1])` # Reverse an array

RANKA
DATE / /
PAGE

~~`[60 50 40 30 20 10]`~~

slicing in a 2D array.

Syntax: `array[row-slice, column-slice]`

`import numpy as np.`

```
arr = np.array([[10, 20, 30, 40],  
               [50, 60, 70, 80],  
               [90, 100, 110, 120]])
```

`print(arr.shape)`

`(3, 4)`

`print(arr[0, :])` → 4 means all the columns.

`[10 20 30 40]`

`print(arr[:2, :])` # Two rows & all columns.

```
[[10 20 30 40]  
 [50 60 70 80]]
```

`print(arr[:, 0])` # selecting columns.

`[10 50 90]`

`print(arr[:, :2])` # first two columns.

```
[[10 20]  
 [50 60]  
 [90 100]]
```

print(arr[0:2, 1:3]) # selecting a subarray

↳ Rows (0 to 1)

→ Columns (1 to 2)

~~[[20 30]
[60 70]]~~

print(arr[1:3, 1:4]) # Rows → 1 to 2

Columns → 1 to 3

~~[[60 70 80]
[100 110 120]]~~

print(arr[:, ::2, :]) # slicing rows with step.

~~[[10 30 40]
[90 110 120]]~~

print(arr[:, :, ::2]) # slicing columns with step.

~~[[10 30]
[50 70]
[90 110]]~~

print(arr[:, ::2, 1:2]) # slicing rows & columns with steps

~~[[10 30]
[90 110]]~~

print(arr[::-1, :]) # Reverse row.

~~[[90 100 110 120]
[50 60 70 80]
[10 20 30 40]]~~

print(arr[:, ::-1]) # Reversing columns.

```
[[ 40  30  20  10]
 [ 80  70  60  50]
 [120 110 100  90]]
```

print(arr[::-1, ::-1]) # Reversing both row & column.

```
[[120 110 100  90]
 [ 80  70  60  50]
 [ 40  30  20  10]]
```

Sllicing in 3D array

Syntax: array [block-slice, row-slice, column-slice]

import numpy as np.

```
arr = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
    [10, 11, 12]])
```

Block 1st
print(arr[0, :, :])
[[1 2 3]
 [4 5 6]]

1st row of each block
print(arr[:, 0, :])

```
[[1, 2, 3]
 [7, 8, 9]]
```

```
print(arr[:, :, :2])
[[[1, 2] [4, 5]]
 [[7, 8] [10, 11]]]
```

Numpy Iteration

Iteration means visiting each element of a Numpy array one by one.

(i) Iterating through 1D array.

import numpy as np.

```
arr = np.array([10, 20, 30, 40])  
for x in arr:  
    print(x)
```

10

20

30

40.

(ii) Iterating through 2D array.

```
arr = np.array([[1, 2, 3],  
                [4, 5, 6]])
```

```
for x in arr:  
    print(x)
```

[1 2 3]
[4 5 6]

```
for row in arr:  
    for x in row:  
        print(x)
```

1
2
3
4
5
6

(iii) Iterating using np.nditer()

```
arr = np.array([[1, 2, 3],  
                [4, 5, 6]])  
for x in np.nditer(arr):  
    print(x)
```

1

2

3

4

5

6

```
arr = np.array([
    [1, 2], [3, 4]]
    [[5, 6], [7, 8]]
])
```

```
for x in np.nditer(arr):
    print(x)
```

1
2
3
4
5
6
7
8

(iv) Iterating using np.ndenumerate()

```
import numpy as np.
```

```
arr = np.array([[10, 20, 30],
                [40, 50, 60]])
```

```
for index, value in np.ndenumerate(arr):
    print(index, value).
```

```
(0, 0) 10
(0, 1) 20
(0, 2) 30
(1, 0) 40
(1, 1) 50
(1, 2) 60
```

(v) Iterating using np.ndindex()

```
arr = np.array([[10, 20],
                [30, 40]])
```

```
for index in np.ndindex(arr.shape):
    print(index, arr[index])
```

```
(0, 0) 10
(0, 1) 20
(1, 0) 30
(1, 1) 40
```

Copying arrays in numpy.

In numpy, copying an array means creating another array from an existing array.

- (i) view $\rightarrow$ does not create separate copy of the data.
- (ii) copy $\rightarrow$ creates a completely independent array.

import numpy as np

arr1 = np.array([10, 20, 30, 40])

* arr2 = arr1 print(arr2) [10 20 30 40]	arr2[0] = 100 print(arr1) # [100 20 30 40] print(arr2) # [100 20 30 40]
---	--

* arr3 = arr1.copy()
~~arr3~~ arr3[0] = 100

print(arr1) # [10 20 30 40]
print(arr3) # [100 20 30 40]

* arr4 = arr1.view()
arr4[0] = 100

print(arr1) # [100 20 30 40]
print(arr3) # [100 20 30 40].

SQL

Database:- It is collection of data in a format that can be easily accessed.

* The software application used to manage our DB is called database management system.

Types of database

Relational

Non-Relational

* Data stored in tables.

* Data not stored in tables.

Ex: MySQL

Ex: MongoDB

What is SQL (Structured Query Language)?

* It is a programming language used to interact with relational database.

* It is used to perform CRUD operations:-

Create

Read

Update

Delete

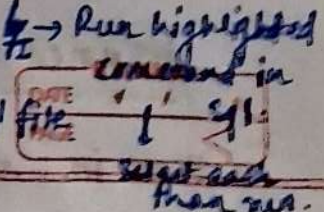
Table:-

Columns → Tells design of the table
Structure / schema

Columns

Rows → Tells individual data

→ Rows

Ctrl + S → To save sql file. { Execute each line separately. } ⚡ → Run highlighted command in Run sql file.  Select each then run.

Creating database :-

CREATE DATABASE db-name; // create database

DROP DATABASE db-name; // delete database.

* CREATE DATABASE college;
USE college;

CREATE TABLE student (
 id INT PRIMARY KEY,
 name VARCHAR(50),
 age INT NOT NULL
);

INSERT INTO student VALUES (1, 'AJAY', 20);
INSERT INTO student VALUES (2, 'AMAN', 19);

SELECT * FROM student; // Print the table.

SQL Datatypes

Datatype	Description	Takes only required memory	Usage
CHAR	store characters of fixed length (0-255)		CHAR(50)
VARCHAR	" " " " " (0-255)		VARCHAR(50)
BLOB	string (0-65535) can store binary large object		BLOB(100)
TINT	integers (-2,147,483,648 to 2,147,483,647)		TINT
TINYINT	integers (-128 to 127)		TINYINT
BIGINT	integers (-9223372036854775808 to ...)		BIGINT
BIT	can store x-bit values (0 ≤ x ≤ 64)		BIT(2)
LOAT	decimal no		LOAT
DATE	Date in format yyyy-mm-dd		DATE
YEAR	year in four digit		YEAR
BOOLEAN	Boolean values 0 or 1		BOOLEAN

Signed and unsigned

RANKA

DATE / /

PAGE

Used in numeric datatypes: INT, FLOAT, DOUBLE, TINYINT

unsigned → stores only +ve no.s

signed → stores +ve & -ve both.

TINYINT UNSIGNED (0 to 255) → Increase range for +ve no.s.

TINYINT (-128 to 127)

Types of SQL Commands:-

DDL (Data definition language):- create, alter, rename, truncate, drop

DQL (Data query language):- select

DML (Data Manipulation language):- insert, update, delete

DCL (Data Control language):- grant & revoke permission to users.

TCL (Transaction Control language):- start transaction, commit, rollback.

CREATE DATABASE IF NOT EXISTS college;

DROP DATABASE IF EXISTS college;

SHOW DATABASES; // show all the databases

SHOW TABLES; // show all the tables.

Reduces chances of error if there is another database with same name

Ques:- Create a database of company XYZ

(i) create a table inside the DB to store employee info
(id, name, salary).

(ii) Add the following information:

1, 'Adam', 25000

2, 'Bob', 30000

3, 'Casey', 40000

(iii) select & view all your table data.

```
CREATE DATABASE CompanyXYZ;
```

```
USE CompanyXYZ;
```

```
CREATE TABLE Companydata (id INT PRIMARYKEY, id  
name VARCHAR(50),  
Salary INT);
```

```
INSERT INTO Companydata (id, name, salary) VALUES  
( 1, 'Adam', 25000,  
2, 'Bob', 30000,  
3, 'Casey', 40000);
```

```
SELECT * FROM Companydata;
```

```
SELECT salary FROM Companydata WHERE salary > 25000;
```

WIDTH 20 20 20 ; // changes width of the table
(in sqLite).

```
SELECT id CURRENT_TIMESTAMP;
```

Keys:-

(i) Primary key:- It is a column (or set of columns) in a table that uniquely identifies each row (a unique id).

There is only one Primary key and it should not be null.

(ii) Foreign key:- It is a column (or set of columns) in a table that refers to the primary key of another table.

Foreign key can have duplicates and null values.

Constraints

SQL constraints are used to specify rules for data in a table.

NOT NULL → columns cannot have null value.

UNIQUE → all values in column are different.

PRIMARY KEY → makes a column unique & not null but used only for one.

FOREIGN KEY → prevents actions that would destroy links between tables.

Ex: `CREATE TABLE temp (cust_id int,
FOREIGN KEY (cust_id) references customer(id)).`

DEFAULT → sets the default value of a column.

Ex: `salary INT DEFAULT 25000;`

CHECK → it can create limit the values allowed in a column.

```
CREATE TABLE newTab (  
    age INT CHECK (age >= 18));
```

```
* CREATE DATABASE collegedata;  
USE collegedata;
```

```
CREATE TABLE student (  
    roll no INT PRIMARY KEY,  
    name VARCHAR(50),  
    marks INT NOT NULL,  
    grade VARCHAR(1),  
    city VARCHAR(20));
```

```
INSERT INTO student (roll no, name, marks, grade, city)  
VALUES (101, "anil", 78, "C", "Pune"),  
       (102, "bhumiika", 93, "A", "Mumbai"),  
       (103, "chetan", 85, "B", "Mumbai"),  
       (104, "dhanu", 96, "A", "Delhi"),  
       (105, "emanuel", 12, "F", "Delhi"),  
       (106, "farah", 82, "B", "Delhi");
```

```
SELECT * FROM student;
```

```
SELECT name, marks FROM student;
```

```
SELECT DISTINCT city FROM student; // Remove duplicate values.
```

```
SELECT * FROM student WHERE marks > 80; } WHERE  
SELECT * FROM student WHERE city = "Mumbai"; } clause.
```

* We can use arithmetic, comparison, logical and bitwise
with WHERE (+, -, *, /, %). (=, !=, >, >=, <, <=) (AND, OR, NOT, IN,
BETWEEN, ALL, LIKE ANY)

SELECT * FROM student WHERE marks > 100;

DATE	/	/
PAGE		

~~SELECT * FROM student WHERE marks > 90 and city = "Mumbai";~~

~~SELECT * FROM student WHERE marks > 90 or city = "Mumbai";~~

~~SELECT * FROM student WHERE marks BETWEEN 80 AND 90;~~
(selects for a given range)

~~SELECT * FROM student WHERE city IN ("Delhi", "Mumbai");~~
(matches any value in the list)

~~SELECT * FROM student WHERE city NOT IN ("Delhi", "Mumbai");~~
(To negate the given condition)

Limit Clause: sets an upper limit on number of (tuples) rows to be returned.

~~SELECT * FROM student LIMIT 3;~~

Order by Clause: To sort in ascending (ASC) or descending (DESC) order.

~~SELECT * FROM student ORDER BY city ASC;~~

~~SELECT * FROM student ORDER BY marks DESC;~~

(Top 3 students) ~~SELECT * FROM student ORDER BY marks DESC LIMIT 3;~~

Aggregate functions: They perform a calculation on a set of values, and return a single value.

COUNT(), MAX(), MIN(), SUM(), AVG().

~~SELECT MAX(marks) FROM student;~~

~~SELECT AVG(marks) FROM student;~~

Group by clause:- Groups rows that have the same values into summary rows.

RANKA	
DATE	/ /
PAGE	

~~It collects data from multiple records and groups the result by one or more column.~~

SELECT City, COUNT(name) FROM STUDENT GROUP BY city;
// count no. of students in each city.

SELECT grade, COUNT(name) FROM student GROUP BY grade;

Having clause:- Applies some conditions on rows (where).
Applies any condition after grouping (having)

SELECT count(name), city FROM student GROUP BY city
HAVING MAX(marks) > 90;

Table related Queries

UPDATE student SET grade = "O" WHERE grade = "A";

UPDATE student SET grade = "B" WHERE ~~grade~~ marks
BETWEEN 80 AND 90;

UPDATE student SET marks = marks + 1;

DELETE FROM student WHERE marks < 33;

Revisiting foreign key:-

```
CREATE TABLE dept(  
    id INT PRIMARY KEY,  
    name VARCHAR(50));
```

```
CREATE TABLE teacher(  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES dept(id);
```

Cascading for foreign key:-

(dept table)

On update Cascade:- When we create a foreign key using `UPDATE CASCADE` the referencing rows are updated in the child table when the referenced row is updated in the parent table which has a primary key.

On delete Cascade:- When we create a foreign key using this option, it deletes the referencing row in the child table when the referenced row is deleted in the parent table which has a primary key.

```
CREATE TABLE 'student'(  
    id INT PRIMARY KEY,  
    courseID INT,  
    FOREIGN KEY (courseID) REFERENCES course (id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE);
```

Table Related queries

Alter (to change the schema).

→ design (column, datatypes, constraints)

DATE	RANKA
NAME	

Add column:- ALTER TABLE student;
ADD COLUMN percentage FLOAT;

Drop column:- ALTER TABLE student;
DROP COLUMN marks;

Rename ^{Table} ~~column~~:- ALTER TABLE student;
RENAMETO studentdata;

change column (rename):- ALTER TABLE student;
CHANGE age stu-age INT;
old dm new dm D-type.
name name

Modify column:- ALTER TABLE student;
~~CHANGE~~ MODIFY age VARCHAR(2);

Truncate:- Delete table's data not the table.

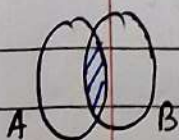
TRUNCATE TABLE student;

DROP student, // Delete the table.

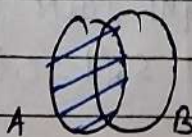
Joins in SQL

Join is used to combine rows from two or more tables, based on a related column between them.

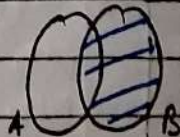
Types of joins:-



Inner join



Left join



Right join



Full join

Outer join

Inner join:- Returns records that have matching values in both tables.

DATE: / /
PAGE:
RANKA
Result

Student

Course

Student-id	name
101	adam
102	bob
103	Casey

Student-id	course
102	eng
105	maths
103	science
107	CSE.

⇒

Student-id	name	Course
102	bob	eng
103	Casey	science

SELECT *

FROM students

INNER JOIN course

ON student.student_id = course.student_id;

Left join:- Returns all records from the left table and the matched records from the right table.

Student

Course.

Result

id	name
101	adam
102	bob
103	Casey

id	Course
102	eng
105	math
103	science
107	CSE

⇒

id	name	Course
101	adam	NULL
102	bob	eng
103	Casey	science

SELECT *

FROM students as s

LEFT JOIN course as c

ON s.id = c.id;

Right join:- Returns all records from the right table, and the matched records from the left table.

Result

id	name	course
102	bob	eng
105	math	math
103	Casey	science
107	NULL	CSE

SELECT *

FROM student as s

RIGHT JOIN course as c

ON s.id = c.id;

Full join:- Returns all records when there is a match in either left or right table.

Result

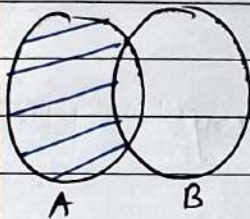
id	name	course
101	adam	null
102	bob	eng
103	casey	science
105	null	maths
107	null	cse

SELECT * FROM ^{student} ~~STUDENT~~ AS a
LEFT JOIN course as b
ON a.id = b.id;

UNION

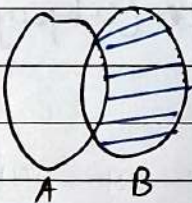
SELECT * FROM student AS a
RIGHT JOIN course AS b
ON a.id = b.id;

Left Exclusive Join:-



SELECT * FROM student AS a
LEFT JOIN course AS b
ON a.id = b.id
WHERE b.id is NULL;

Right Exclusive Join:-



SELECT * FROM student AS a
RIGHT JOIN course AS b
ON a.id = b.id
WHERE a.id is NULL;

Self Join:- It is a regular join by table is joined with itself.

SELECT *
FROM student as s
JOIN course as c
ON s.id = c.id.

union: It is used to combine the result set of two or more SELECT statements.

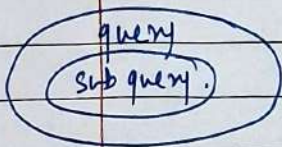
- * every SELECT should have same no. of columns.
- * columns must have same data types.
- * Columns in every select should be in same order.

```
SELECT column(s) FROM tableA  
UNION  
SELECT column(s) FROM tableB
```

UNION ALL → Executes even the duplicates that are in both tables.

SQL subqueries

- It is an inner query within the SQL query.
- It involves 2 select statements



```
SELECT column(s)  
FROM table-name  
WHERE col-name operator (subquery);
```

Ex: (i) Find avg. of class

(ii) Find the name of student with marks > avg.

(iii) Find students of Delhi

(iv) Find their max marks using the sublist in step (iii)

roll no.	name	marks	city
101	anil	78	Pune
102	bhumnika	93	Mumbai
103	chetan	85	Mumbai
104	dhruv	96	Delhi
105	emanuel	92	Delhi
106	farah	82	Delhi

```
{ SELECT AVG (marks) FROM student;
```

```
{ SELECT names FROM student WHERE
```

RANKA		
DATE	/	/
PAGE	marks > 87.6667;	

```
SELECT * FROM student WHERE city = "Delhi";
```

```
→ SELECT MAX (marks)
```

```
FROM ( SELECT * FROM student WHERE city = "Delhi" ) AS temp;
```

MySQL views

A view is a virtual table based on the result-set of an SQL statement.

A view always shows up-to-date data. The database engine recreates the view, everytime a user queries it.

```
CREATE VIEW view1 AS
```

```
SELECT roll no, name FROM student;
```

```
SELECT * FROM view1;
```